



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 01

NOMBRE COMPLETO: GARCÍA SITIO DIEGO

N° de Cuenta: 318120601

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 29/08/2026

CALIFICACIÓN: _____

EJERCICIO 1

Descripción: En este ejercicio se debe cambiar el color de fondo de la pantalla entre rojo, verde y azul de forma cíclica y solamente mostrando esos 3 colores con un periodo de lapso adecuado para el ojo humano.

Solución: En la *ilustración 1* está el código que cambia el fondo cada segundo con los colores solicitados. Se utilizaron condicionales para poder usar los segundos que se leen con “glfwGetTime()” y dar el efecto de “reloj” en la ejecución del programa. Además, se comentó la línea de código “glClearColor(0.0f,0.0f,0.0f,1.0f);” para evitar que limpiara lo que dibujábamos en el Canvas.

De esta forma, en la *ilustración 2* se muestra una captura de cómo cambia el color del fondo, además de mostrar el triángulo que dibujó el profesor.

```
166 //Loop mientras no se cierra la ventana
167 while (!glfwWindowShouldClose(mainWindow))
168 {
169     //Recibir eventos del usuario
170     glfwPollEvents();
171
172
173     // Creamos una variable de tiempo para obtener el tiempo en segundos
174     int tiempo = (int)glfwGetTime();
175
176     // Usamos el operador módulo para alternar entre 0, 1 y 2 cada segundo
177     if (tiempo % 3 == 0)
178     {
179         glClearColor(1.0f, 0.0f, 0.0f, 1.0f); // Rojito
180     }else if (tiempo % 3 == 1)
181     {
182         glClearColor(0.0f, 1.0f, 0.0f, 1.0f); // Verdecito
183     }else
184     {
185         glClearColor(0.0f, 0.0f, 1.0f, 1.0f); // Azulito
186     }
187
188     glClear(GL_COLOR_BUFFER_BIT);
189
190
191     //Limpiar la ventana
192     /* Comentamos esta parte del código para que no limpie lo que dibujamos
193     glClearColor(0.0f,0.0f,0.0f,1.0f);
194     */
195 }
```

Ilustración 1: Condicionales para crear el "reloj".

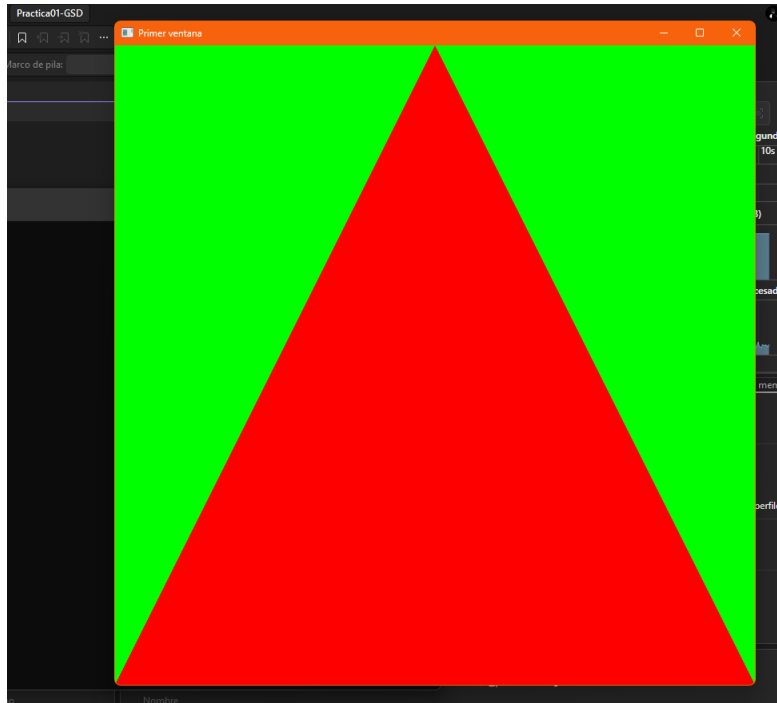


Ilustración 2: Captura de pantalla del ejercicio 1 en ejecución.

Problemas presentados: No era muy claro cómo hacer el reloj en C++, así que pregunté cómo poder hacerlo, de esta forma me ayudó a entender mejor cómo funcionaba el módulo.

EJERCICIO 2

Descripción: En este ejercicio se debe dibujar de forma simultánea en la ventana 1 rombo y 1 trapecio separados.

Solución: En la *ilustración 3* está el código que coloca los vértices de ambas figuras, recordando que tenemos 2 unidades espaciales de arriba hacia abajo y de izquierda hacia la derecha. Con esto en mente, colocamos los puntos correspondientes. Con lo anterior realizado, en la *ilustración 4* se muestra la línea de código 219 con el número 15, que representa el número total de vértices (de ahí que en la *ilustración 3* estén las multiplicaciones de la cantidad total de vértices de cada figura).

Para finalizar, en la *ilustración 5* se observa cómo cambia el fondo mientras se visualizan las figuras solicitadas.

```

32 void CrearTriangulo()
33 {
34     GLfloat vertices[] = {
35         //Colocamos los vertices para el rombo (3*2)
36         -0.5f,  0.5f, 0.0f,
37         -0.8f,  0.0f, 0.0f,
38         -0.5f, -0.5f, 0.0f,
39
40         -0.5f,  0.5f, 0.0f,
41         -0.5f, -0.5f, 0.0f,
42         -0.2f,  0.0f, 0.0f,
43
44         //Colocamos los vertices para el trapecio (3*3)
45         0.2f, -0.5f, 0.0f,
46         0.5f, -0.5f, 0.0f,
47         0.35f, 0.5f, 0.0f,
48
49         0.35f, 0.5f, 0.0f,
50         0.5f, -0.5f, 0.0f,
51         0.65f, 0.5f, 0.0f,
52
53         0.5f, -0.5f, 0.0f,
54         0.8f, -0.5f, 0.0f,
55         0.65f, 0.5f, 0.0f
56     };
57     glGenVertexArrays(1, &VAO); //generar 1 VAO

```

Ilustración 3: Código que contiene los vértices para dibujar las dos figuras.

```

218 //((3*2+3*3 = 15)
219 glDrawArrays(GL_TRIANGLES,0,15);

```

Ilustración 4: Código para agregar la cantidad total de vértices.}

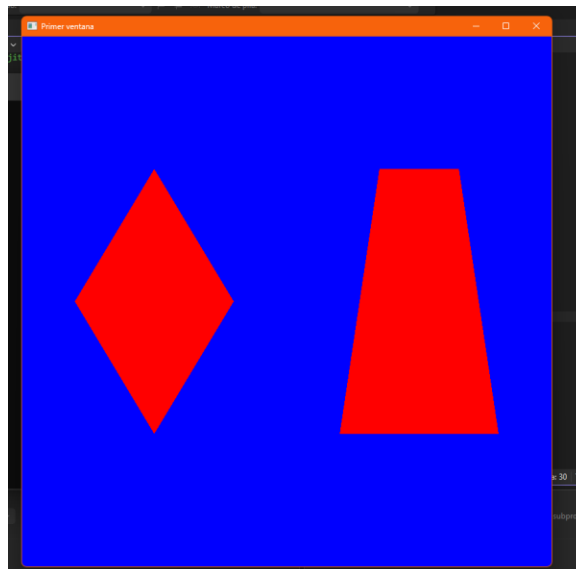


Ilustración 5: Captura de pantalla del ejercicio 2 en ejecución.

Problemas presentados: El problema principal es entender cómo dibujar las figuras por medio de puntos. De ahí en fuera, todo es prueba y error

CONCLUSIÓN

El ejercicio 2 es un poco complicado de entender, siento que sería mejor que el profesor intentara dibujar una figura muy complicada para que después nos pida hacer una fácil, como un cuadrado. Y tal vez en la práctica pida realizar estas figuras. Respecto al ejercicio 1, sería bueno que nos diera el código hecho del “reloj”, con la intención de poco a poco ir recordando el lenguaje C++, permitiendo que en la práctica se nos pidan otras cosas un poco más complejas para ir recordando o volviendo aprender los conocimientos olvidados de semestres anteriores. Fuera de estos comentarios, los ejercicios son interesantes.